

Aula 16 - DCL e Segurança de Dados

Descrição: Implementar controle de acesso e segurança utilizando comandos DCL (GRANT, REVOKE) e garantir a integridade referencial dos dados.

Conteúdo

DCL (Data Control Language): GRANT e REVOKE

A sublinguagem DCL é a parte do SQL responsável pela segurança, controle de acesso e autorização sobre os objetos do banco de dados (como tabelas, views, procedures). Ela atua definindo o que cada usuário ou perfil (**role**) está autorizado a executar no sistema. Os comandos centrais que formam o núcleo do controle de acesso são:

- **GRANT:** É o comando utilizado para conceder (atribuir) privilégios de acesso a um usuário ou grupo sobre um objeto específico. Com ele, podemos liberar operações específicas como ler dados (**SELECT**), inserir (**INSERT**), atualizar (**UPDATE**) ou deletar (**DELETE**) registros.
- **REVOKE:** É o comando que atua na direção oposta, removendo privilégios que foram concedidos anteriormente. É essencial para corrigir concessões erradas, reduzir o nível de acesso de um usuário que mudou de função ou remover acessos que não são mais necessários.

Princípio do Menor Privilégio

Este é um dos conceitos mais importantes na segurança de dados: consiste em dar a cada usuário **apenas e exatamente** o nível mínimo de permissões necessárias para que ele realize o seu trabalho.

- Na prática, isso significa evitar o uso indiscriminado de concessões amplas, como conceder **ALL PRIVILEGES** a usuários comuns, e estruturar a separação de funções (por exemplo, um usuário de aplicação tem permissões diferentes de um usuário de relatórios).

Integridade Referencial

A integridade referencial é a garantia de que os relacionamentos entre tabelas conectadas por chaves estrangeiras (**FOREIGN KEY**) permaneçam consistentes ao longo do tempo.

- O seu principal papel é assegurar que nenhum registro em uma tabela "filha" fique órfão, ou seja, apontando para um registro na tabela "pai" que já não existe mais.
- Ela atua limitando ou determinando o comportamento do banco de dados quando os registros nas tabelas "pai" sofrem operações de atualização (**UPDATE**) ou exclusão (**DELETE**).

Ações de Chave Estrangeira (ON DELETE / ON UPDATE)

Quando estabelecemos uma chave estrangeira, precisamos dizer ao banco de dados como ele deve reagir se um registro pai for apagado ou modificado. As principais ações suportadas são:

- **RESTRICT (Bloqueia):** Impede a exclusão ou a atualização de um registro na tabela pai caso existam registros filhos relacionados a ele. Ao tentar a ação, o banco de dados bloqueia a operação e lança um erro de violação de chave estrangeira.
- **CASCADE (Propaga):** Propaga a alteração. Se o registro pai for excluído ou tiver sua chave atualizada, a mesma operação (exclusão ou atualização) acontecerá automaticamente e de forma transparente em todos os registros filhos relacionados, evitando que fiquem órfãos.
- **SET NULL (Anula o vínculo):** Permite que o registro pai seja alterado ou excluído. Como consequência, o banco de dados define o campo correspondente (a chave estrangeira) na tabela filha como nulo (**NULL**), mantendo o registro filho, mas desvinculando-o do registro pai removido.

Atividades Práticas

Para esta atividade utilizaremos o Banco de Dados Sakila.

Atividade 1: Criação de Perfis e Controle de Acesso (DCL)

Nesta atividade, simularemos a criação de dois funcionários da locadora com responsabilidades distintas.

1. Criar Usuários:

SQL

```
CREATE USER 'atendente_loja'@'localhost' IDENTIFIED BY 'senha123';  
CREATE USER 'analista_financeiro'@'localhost' IDENTIFIED BY 'senha456';
```

2. Conceder Permissões (Princípio do Menor Privilégio):

- O **atendente** deve apenas realizar aluguéis e consultar filmes.
- O **analista** deve apenas auditar pagamentos.

SQL

-- Atendente: consulta e inserção de aluguéis

```
GRANT SELECT, INSERT ON sakila.rental TO 'atendente_loja'@'localhost';  
GRANT SELECT ON sakila.film TO 'atendente_loja'@'localhost';
```

-- Analista: apenas leitura de pagamentos

```
GRANT SELECT ON sakila.payment TO 'analista_financeiro'@'localhost';
```

3. Teste de Violação de Segurança:

Para executar o teste de violação de segurança, você precisará tentar apagar um registro (DELETE) da tabela `payment` logado como `analista_financeiro`. (usuário restrito). Como resultado, o sistema deve negar a operação. Aqui está o passo a passo e o comando exato:

- Conectando com o usuário correto no DBeaver

Para que o teste funcione, o DBeaver precisa estar conectado com o usuário que criamos, e não com o usuário administrador (root).

1. No DBeaver, crie uma nova conexão MySQL.
2. Nas credenciais, preencha:
 - a. **Username**: `analista_financeiro`
 - b. **Password**: `senha456`
 - c. Clique na aba **Propriedades do Driver** (Driver Properties).
 - d. Na lista de propriedades que vai aparecer, procure por **`allowPublicKeyRetrieval`**.
 - e. Altere o valor dessa propriedade de **`FALSE`** para **`TRUE`**.
 - i. **Dica**: Se você não achar na lista, pode clicar no botão de "+" (Add) para adicionar a propriedade manualmente com o nome **`allowPublicKeyRetrieval`** e o valor **`TRUE`**.
3. Conclua a criação e conecte-se.

- O Comando de Teste

Abra um novo **Editor SQL** (SQL Editor) nesta nova conexão do `analista_financeiro` e execute o seguinte comando:

SQL

```
DELETE FROM sakila.payment WHERE payment_id = 1;
```

Resultado Esperado

Como concedemos apenas a permissão de `SELECT` (leitura) para este usuário na tabela `payment`, o DBeaver não conseguirá executar a exclusão. Ele retornará um erro de permissão negada no painel de resultados, que será algo muito semelhante a isso:

Erro SQL [1142] [42000]: *DELETE command denied to user 'analista_financeiro'@'localhost' for table 'payment'*

Isso prova que o DCL (Princípio do Menor Privilégio) está funcionando perfeitamente na prática!

Atividade 2: Integridade Referencial na Prática (FK Actions)

Vamos analisar o comportamento das chaves estrangeiras na relação `customer` (Pai) e `rental` (Filho). Para esta atividade, você deve voltar para o usuário **root (Administrador)**.

1. Analisar a Constraint Atual:

- Verifique como o banco reage ao tentar excluir um cliente que possui aluguéis ativos. (O padrão costuma ser `RESTRICT`).

2. Alterar Comportamento para Segurança de Dados:

- Cenário:** Se um cliente for excluído por erro, não queremos perder o histórico de aluguéis, mas os registros não podem ficar "órfãos". Vamos usar `SET NULL`.
- Você vai pedir para o banco de dados colocar o valor `NULL` na coluna `customer_id` da tabela `rental` caso o cliente seja deletado. Porém, na estrutura original do banco *sakila*, a coluna `customer_id` na tabela `rental` foi criada com a regra **NOT NULL** (não aceita valores nulos).

Para que a nossa regra funcione, primeiro precisamos alterar a estrutura da coluna para permitir valores nulos. Utilize os comandos abaixo:

SQL

```
-- Removendo a FK atual e recriando com SET NULL
```

```
ALTER TABLE sakila.rental MODIFY customer_id SMALLINT UNSIGNED NULL;
```

```
ALTER TABLE sakila.rental
```

```
ADD CONSTRAINT fk_rental_customer
```

```
FOREIGN KEY (customer_id) REFERENCES customer(customer_id)
```

```
ON DELETE SET NULL ON UPDATE CASCADE; --
```

3. Teste de Integridade:

- Exclua um cliente de teste e observe se o `customer_id` na tabela `rental` tornou-se `NULL` automaticamente.

Vamos prosseguir com a exclusão do cliente para ver a mágica acontecendo:

SQL

```
-- Verificar os aluguéis do cliente 1
```

```
SELECT * FROM sakila.rental WHERE customer_id = 1;
```

```
-- Deletar os pagamentos dele primeiro (pois a tabela payment também segura o cliente)
```

```
DELETE FROM sakila.payment WHERE customer_id = 1;
```

```
-- Deletar o cliente
```

```
DELETE FROM sakila.customer WHERE customer_id = 1;
```

```
-- Verificar a tabela de aluguéis de novo (os registros ainda estarão lá, mas com customer_id = NULL)
```

```
SELECT * FROM sakila.rental WHERE customer_id IS NULL;
```

4. Resultado esperado:

O resultado esperado deste teste é a comprovação visual de que a regra **ON DELETE SET NULL** funcionou perfeitamente, mantendo o histórico de dados sem gerar inconsistências (registros órfãos).

Aqui está o que vocês verão acontecer passo a passo:

1. **SELECT * FROM sakila.rental WHERE customer_id = 1;**
 - **Resultado:** O DBeaver mostrará uma tabela com vários registros de alugueis (geralmente mais de 30 no banco Sakila padrão) onde a coluna `customer_id` está preenchida com o número **1**.
2. **DELETE FROM sakila.payment WHERE customer_id = 1; e DELETE FROM sakila.customer WHERE customer_id = 1;**
 - **Resultado:** Os comandos serão executados com sucesso (mensagem de *OK* ou *Rows affected*). O banco **não** bloqueará a exclusão do cliente, pois a regra restritiva foi substituída.
3. **SELECT * FROM sakila.rental WHERE customer_id IS NULL;**
 - **Resultado Principal:** A tela de resultados mostrará **exatamente os mesmos alugueis** que apareceram no passo 1. A grande diferença – e a "mágica" da aula – é que agora a coluna `customer_id` de todas essas linhas estará preenchida com **NULL** (um valor nulo/vazio).

Conclusão:

O teste prova que o SGBD fez a manutenção automática da Integridade Referencial. O cliente foi excluído do sistema, mas a locadora **não perdeu o seu histórico de alugueis** (o que aconteceria se usassem **CASCADE**) e o sistema **não travou a exclusão** (o que aconteceria no **RESTRICT**). O vínculo foi simplesmente desfeito com segurança.

Atividade 3: Cenário Integrado (Segurança + Integridade)

É a hora de mostrar que a proteção de um banco de dados é feita em camadas e que o DCL e a Integridade Referencial são melhores amigos na vida real de um DBA.

Objetivo: Demonstrar que ter permissão de acesso (DCL) não dá ao usuário o direito de quebrar as regras de negócio e a consistência do banco de dados (Integridade Referencial).

1. O Contexto e o Problema

Relembre os alunos da Atividade 1: nós demos ao usuário `atendente_loja` a permissão de **INSERT** na tabela `rental`. Ou seja, o crachá dele permite entrar no sistema e registrar um novo aluguel.

- **O Problema:** E se esse atendente digitar o código de um filme/inventário errado na hora de alugar? O sistema vai aceitar só porque ele tem permissão de **INSERT**?

2. O Desafio Prático

Vamos nos conectar com o usuário `atendente_loja` e tentar registrar um aluguel para um item de inventário que não existe (por exemplo, `inventory_id = 999999`).

Comando para execução:

SQL

-- Tentativa de inserir um aluguel com um filme que não existe no inventário

INSERT INTO sakila.rental (rental_date, inventory_id, customer_id, staff_id)

VALUES (NOW(), 999999, 1, 1);

3. Resultado Esperado

Vai retornar um erro, mas **não será um erro de permissão negada (DCL)** como vimos na Atividade 1. Será um erro de chave estrangeira!

Erro SQL [1452] [23000]: *Cannot add or update a child row: a foreign key constraint fails (sakila.rental, CONSTRAINT fk_rental_inventory FOREIGN KEY (inventory_id) REFERENCES inventory (inventory_id))*

Conclusão

Nesta aula demonstramos que um Banco de Dados robusto não depende apenas de tabelas bem estruturadas, mas de uma gestão ativa de **quem** pode acessar os dados e de **como** esses dados se comportam em caso de alterações. A combinação de permissões restritivas (DCL) com regras de integridade (FK Actions) cria um ambiente onde o erro é interceptado pelo sistema antes de comprometer a base de dados.

Na Próxima Aula

Enquanto nesta aula focamos em evitar a corrupção lógica e acessos indevidos, a **Aula 17** tratará da última linha de defesa: a recuperação física dos dados em caso de falha catastrófica ou perda total.